Cloud Solutions: Project Draft

Team: Carla Escassi, Marc Farràs, Laura Rigau, Marta Palet

1. Project idea: SmartFuel Forecast

Fuel prices in Spain change a lot from one day to another and between stations that are very close to each other. Our idea is to build a tool that tells you if it is better to fill up the tank today or wait, and which station near you (or on your route) is the cheapest. Registered users can also follow their usual stations and receive an email alert when the price drops below a target.

To do this we will use the open data from the Spanish Ministry for the Ecological Transition, which publishes the prices of around 12,000 petrol stations several times a day. We will collect this data automatically, save the history and use it to train a model that predicts if prices will go up or down in the next days. Since the data keeps arriving all the time and grows every day, we think it is a good fit for a serverless, event-driven cloud solution instead of doing it manually.

2. How we plan to build it (AWS)

- **Data collection:** an EventBridge schedule (used as a cron job) will launch a Lambda function four times a day. This Lambda calls the Ministry API (one call returns all the stations) and saves the raw JSON in S3.
- **Processing (Step Functions):** when a new file appears in the S3 bucket, an EventBridge rule starts a Step Functions workflow. The workflow has these steps:
    - **Validate:** a Lambda checks the file (format and number of stations). If the file is not valid, it is moved to a quarantine folder and the admins receive an email through SNS.
    - **Clean:** another Lambda cleans the data, saves it in S3 in Parquet format and finds which prices changed since the previous snapshot.
    - **Save and predict:** two steps run in parallel. One saves the changed prices in DynamoDB, and the other loads the model from S3 and saves the predictions.
    - **Error handling:** every step has automatic retries and error handling. Events that cannot be delivered go to an SQS dead-letter queue so we can check them later.
- **Price alerts (DynamoDB Streams):** when a price changes in the DynamoDB table, DynamoDB Streams triggers a Lambda. This Lambda checks which users follow that station. If the price drops below their target or changes a lot, it sends them an email with Amazon SES. This way the alerts are completely event-driven and separated from the processing workflow.
- **Model retraining (Step Functions):** the first model will be trained on our laptops and uploaded to S3. After that, a second workflow runs every week:
    - It builds the training data with an Athena query over the history.
    - It trains a new simple model in a Lambda.
    - It compares the new model with the current model and with a basic baseline ("the price stays the same").

    - Only if the new model is better is it saved as the production model. If not, we keep the old one and the admins are notified.
- **Storage:**
    - **S3** will be our data lake with all the historical prices (raw data, clean data and models). We will query it with Athena, which works directly on the S3 buckets without any server. Lifecycle policies will move old raw files to a cheaper storage tier.
    - **DynamoDB** (on-demand mode) will store what the web app needs quickly: the stations (with a geohash index to search by area), the latest prices, the predictions and the users' alert subscriptions.
- **Web app:** a static website with a map of the stations, hosted on AWS Amplify. It gets the data from an HTTP API built with API Gateway and Lambda. API Gateway will also limit the number of requests (throttling) and cache the most common responses to reduce the load on the backend.
- **Authentication:** users will sign up and log in with Amazon Cognito User Pools. The map and the predictions are public, but following stations and receiving alerts requires an account, and the admin part is only for users in the admin group. API Gateway checks the Cognito token (JWT) before calling the Lambdas.
- **Security:** each Lambda and workflow will have its own IAM role with only the permissions it needs (minimum privilege). The S3 buckets will not be public, and the data will be encrypted at rest and in transit. We will use CloudWatch for logs, metrics and alarms. Since the whole solution is serverless and we do not use servers or a relational database, we do not need a VPC: all the services are accessed through AWS with IAM permissions.

3. Budget and limitations

We have a budget of around $50. Since the whole solution is serverless (no EC2, RDS, load balancer or NAT Gateway), we only pay for what we use, and nothing costs money when it is not running. With our volume of data, Lambda, Step Functions, API Gateway, Cognito and Amplify should stay almost free. DynamoDB will be cheap because we only save the prices that changed, not all the stations every time, and S3 and Athena should cost a few cents. We will also create a budget alert in AWS to avoid surprises.

Some limitations we already see:

- **Little history:** we can only train with the data we collect from now on, so the model will be simple at the beginning and the data collection has to start as soon as possible.
- **Prediction quality:** prices also depend on things outside our data (oil price, taxes), so the predictions will not be perfect.
- **Lambda limits:** a Lambda can run for a maximum of 15 minutes and has limited memory (up to 10 GB), so the model must be light.
- **Searching by location:** DynamoDB is not as flexible as SQL for this, so we will use a geohash index.
- **Emails:** at the beginning SES only sends emails to verified addresses, which is enough for the demo.